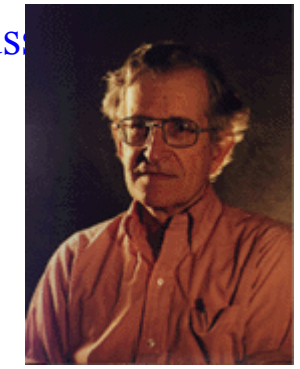
Syntaxbeschreibung von Programmiersprachen

Beschreibung syntaktisch korrekter Programme

- ***Syntax:*** Regeln der richtigen Satzbildung
  - gibt an, wie die Sätze (Programme) einer Sprache aufgebaut sein müs
  - noch keine Aussage über die Semantik!



N. Chomsky

- Programmiersprachen sind ***formale Sprachen***
  - einfacher und präziser definiert als natürliche Sprache
    - überblickbarer, endlicher Zeichenvorrat
    - endliche Regelmenge
- Syntax formaler Sprachen wird durch ***Grammatiken*** beschrieben
  - beschreiben Sätze einer Sprache durch (Produktions-) Regeln
  - eine formale Sprache (Metasprache), die eine formale Sprache spezifiziert
  - hier: eine (Meta-) Sprache die alle ***syntaktisch korrekten Programme*** in einer Programmiersprache spezifizieren kann
- Klassen von Grammatiken (***Chomsky-Hierarchie***)
  - reguläre, kontextfreie, kontextsensitive, allgemeine Grammatiken
  - wichtig für die Definition von Programmiersprachen sind ***kontextfreie Grammatiken***

Einführendes Beispiel: Satzaufbau der deutschen Sprache

- **einfacher Satz besteht aus Subjekt, Prädikat und Objekt**

  (1) Satz = Subjekt Prädikat Objekt ".". — Produktionsregel

- **Bestandteile eines Satzes können durch weitere Regeln beschrieben werden**

  (2) Subjekt = Artikel Substantiv.
  (3) Artikel = "Die".
  (4) Substantiv = "Maus" | "Katze". — Nichtterminalzeichen
  (5) Prädikat = "frißt".
  (6) Prädikat = "haßt". — Meta-Operator
  (7) Objekt = "Käse".

  Terminalzeichen

Einführendes Beispiel: Ableitung in der Sprache

- **nach diesen Regeln können korrekte Sätze konstruiert werden**

Satz → Subjekt Prädikat Objekt "."	(Regel 1)
→ **Artikel Substantiv** Prädikat Objekt "."	(Regel 2)
→ **"Die"** Substantiv Prädikat Objekt "."	(Regel 3)
→ "Die" Substantiv Prädikat **"Käse"** "."	(Regel 7)
→ "Die" **"Maus"** Prädikat "Käse" "."	(Regel 4)
→ "Die" "Maus" **"frißt"** "Käse" "."	(Regel 5)

- **Konstruktion eines Satzes aus den Produktionsregeln nennt man eine Ableitung in der Sprache**

Kontextfreie Grammatiken

Syntax von Programmiersprachen: Kontextfreie Grammatik

$G = (N, T, P, S)$, mit $N \cap T = \Phi$

- **T - Terminalzeichen**
  - eine endliche, nichtleere Menge von Zeichen oder Zeichenfolgen, die der zu definierenden (Programmier-)Sprache angehören
  - jedes syntaktisch korrekte Programm ist eine Folge von Terminalzeichen z.B. "if" "while" "1" "int" etc.
- **N - Nichtterminalzeichen, syntaktische Variablen**
  - ist eine endliche, nichtleere Menge von Zeichen, die bestimmte syntaktische Konstrukte (Programmkomponenten) darstellen
  - werden schrittweíse ersetzt, bis nur noch Terminalsymbole übrig sind z.B. Declaration, Type, Name, Block, Stmts, etc.
- **P - Produktionsregeln, Ersetzungsregeln**
  - endliche Menge von Paaren der Form p = q
  - p aus N und q aus $(N \cup T)^*$
  - legen fest, wie man aus bereits bekannten Konstrukten durch Ersetzung neue Konstrukte erhält
  - „Operatoren“ der Produktionsregeln werden Metasymbole genannt
- **$S \in N$ ist das Startsymbol**

Kontextfreie Grammatik: Beispiel

Grammatik MK = (N,T, P, S)

N = {Satz, Subjekt, Prädikat, Objekt, Artikel, Substantiv}

T = {"Die", "Maus", "Katze", "frißt", "haßt", "Käse", "."}

P = {

Satz = Subjekt Prädikat Objekt .

Subjekt = Artikel Substantiv

Artikel = "Die"

Substantiv = "Maus" | "Katze"

Prädikat = "frißt"

Prädikat = "haßt"

Objekt = "Käse"

}

S = Satz



- kontextfreie Grammatiken

Produktionsregeln enthalten auf der linken Seite genau ein Nichtterminalzeichen

Kontextfreie Grammatik: Ableitung & Erzeugte Sprache

Sei G = (N,T,P,S), sowie y, y', f ∈ (N ∪ T)* und p=q ∈ P gegeben:

- wir nennen y' eine ***direkte Ableitung aus y (bzgl. p=q)***, wenn y' durch einmalige Anwendung von p=q aus y entsteht

Bemerkung: ist y' ***direkt ableitbar aus y (bzgl. p=q)*** **(y → y')**, dann muss gelten:

- y ist als ***xpz*** und y' ist als ***xqz*** darstellbar , x, z ∈ (N ∪ T)*

- f heißt ***ableitbar aus y bezüglich G*** **(y $\rightarrow^*_G$ f)**, wenn f durch die sukzessive Anwendung von direkten Ableitungen aus y entsteht

Kontextfreie Grammatik: Ableitung & Erzeugte Sprache

Sei G = (N,T,P,S) gegeben:

- die von ***G erzeugte Sprache L(G)*** ist die Menge aller Wörter
  - die aus dem Startsymbol ableitbar sind
  - und nur aus Terminalsymbolen bestehen

$$\mathbf{L(G) = \{w \in T^* \mid S \rightarrow^*_G w\}}$$

- v ∈L(G) nennt man auch ***Satz***, v' ∈ (N ∪ T)* eine ***Satzform***

Beispiel: Grammatik MK

L(MK) = {"Die" "Maus" "frißt" "Käse" ".",
"Die" "Maus" "haßt" "Käse" ".",
"Die" "Katze" "frißt" "Käse" ".",
"Die" "Katze" "haßt" "Käse" "." }

Satz $\rightarrow^*_{MK}$ "Die" "Katze" "haßt" "Käse" ".", wegen

Satz → Subjekt Prädikat Objekt "."
→ **Artikel Substantiv** Prädikat Objekt "."
→ **"Die"** Substantiv Prädikat Objekt "."
→ "Die" Substantiv Prädikat **"Käse"** "."
→ "Die" **"Katze"** Prädikat "Käse" "."
→ "Die" "Katze" **"haßt"** "Käse" "."

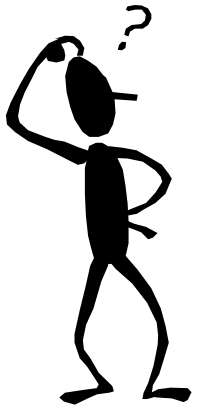
etc.

Beispiel: Natürliche Zahlen

Ein erster Versuch:

Grammatik NAT = (N, T, P, natürliche-Zahl), mit

N = {Ziffer, natürliche-Zahl}
T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
P = {
natürliche-Zahl = Ziffer
natürliche-Zahl = Ziffer Ziffer
natürliche-Zahl = Ziffer Ziffer Ziffer
....
Ziffer = 0 |1 |2 |3 |4 |5 |6 |7 |8 |9
}



Beschreibung unendlicher Sprachen

L(NAT) beschreibt zwar die Menge der natürlichen Zahlen, aber P enthält unendliche Anzahl an Produktionsregeln.

Beispiel: Natürliche Zahlen

Grammatik NAT = (N, T, P, natürliche-Zahl), mit

N = {Ziffer, natürliche-Zahl}

T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

P = {

natürliche-Zahl = Ziffer | Ziffer natürliche-Zahl

Ziffer = 0 |1 |2 |3 |4 |5 |6 |7 |8 |9

}

L(NAT) beschreibt Darstellung der natürlichen Zahlen

z.B. 352 ϵ L(NAT), wegen

natürliche-Zahl → Ziffer natürliche-Zahl

→ **3** natürliche-Zahl

→ 3 **Ziffer natürliche-Zahl**

→ 3 **5** natürliche-Zahl

→ 3 5 **Ziffer**

→ 3 5 **2**



Unendliche Sprachen:
Verwendung von rekursiv definierten Produktionsregeln

Beispiel: Bezeichner in Programmiersprachen

Grammatik Bezeichner = (N, T, P, Name), mit

N = {Name, Buchstabe, Ziffer}

T = {a, ..., z, A, ..., Z, 0, ..., 9}

P = {

Name = Buchstabe | Name Buchstabe | Name Ziffer.

Buchstabe = a | b | ... | z | A | B | ... | Z

Ziffer = 0 |1 |2 |3 |4 |5 |6 |7 |8 |9

}

L(Bezeichner) beschreibt Namen von Bezeichnern einer Programmiersprache

z.B. a1 ϵ L(Bezeichner), wegen

Name → Name Ziffer

→ **Buchstabe** Ziffer

→ Buchstabe **1**

→ **a**1

EBNF – Extended Backus-Naur-Form

EBNF- Extended Backus-Naur-Form

- vereinfachte Beschreibung von kontextfreien Sprachen
  - Standard zur Beschreibung von Programmiersprachen
- ***EBNF*** (Wirth 1971)
  - BNF: Backus-Naur-Form (entwickelt 1960 für Algol-60)
  - Erweiterte BNF (EBNF) gestattet bessere Handhabung
- kompakter und leichter leserlich als Grammatiken
  - textbasiert
  - zugrundeliegende Grammatik wird im wesentlichen durch Produktionsregeln beschrieben
  - Einführung zusätzlicher Metasymbole zur Beschreibung der rechten Seite einer Produktionsregel

EBNF – Eigenschaften

- Produktionsregeln werden in der Form p = q geschrieben
  - mit einem „ .“ abgeschlossen

- Terminalzeichen werden zwischen " … " geschrieben
- Nichtterminalzeichen werden nicht in " … " geschrieben

- Startsymbol ist linke Seite der ersten Produktionsregel

- ***Metasymbole*** in den Produktionsregeln (rechte Seite der Regel):
  - **x | y** ... ***Alternative***: entweder x oder y
  - [x] ... ***Option***: x gar nicht oder einmal
  - { x } ... ***Wiederholung***: x gar nicht oder beliebig oft
  - (x) ... Gruppe: fasst eine Menge von Symbolen zusammen

Beispiel: Wiederholungen in EBNF

natürliche-Zahl = Ziffer | Ziffer natürliche-Zahl.

in EBNF:

natürliche-Zahl = Ziffer {Ziffer}.
Ziffer = "1" | "2" | "3" | "4" | "5"| "6" | "7" | "8" | "9" | "0".

Name = Buchstabe | Name Buchstabe | Name Ziffer.

in EBNF:

Name = Buchstabe {Buchstabe | Ziffer}.
Buchstabe = "a" | "b" | ... | "z" | "A" | "B" | ... | "Z".
Ziffer = "1" | "2" | "3" | "4" | "5"| "6" | "7" | "8" | "9" | "0".



Metasymbol {} kann in EBNF anstelle einer rekursiven Darstellung verwendet werden!!!!!

Beispiel: Ganzzahlige Konstanten in C

IntegerLiteral = ["+" | "-"] (OctLiteral | DeciLiteral | HexLiteral)
["u" | "U"| "l" | "L" | "ul" | "UL"| … | "ll" | "LL"].

OctLiteral = "0" OctDigit {OctDigit}.

OctDigit = "0" | ... | "7".

DeciLiteral = Digit1 {Digit} | "0".

Digit1 = "1" | ... | "9".

Digit = "0" | Digit1.

HexLiteral = ("0x" | "0X") HexDigit {HexDigit}.

HexDigit = Digit | "a" | "A" | ... | "f" | "F".